

Snake — Engine Rebuild Completion Status

Date: 23 July 2026 **Scope:** Top-down assessment of how far the `game-with-engine-architecture` branch is from reproducing the working game on `origin/simple-game`, and what's left to get there. **Sources:** `dev-log/2026-07-08-snake.md`, `dev-log/2026-07-22-snake.md`, `dev-log/2026-07-23-snake.md`; a live build attempt on this branch (see below); direct reading of current source; `origin/simple-game` (`src/main.cpp`) as the feature-complete reference.

TL;DR

The game does not build, and even once it does, none of the core gameplay loop is wired to anything that runs. This isn't "a few bugs from working" — it's two separate, sequential gaps:

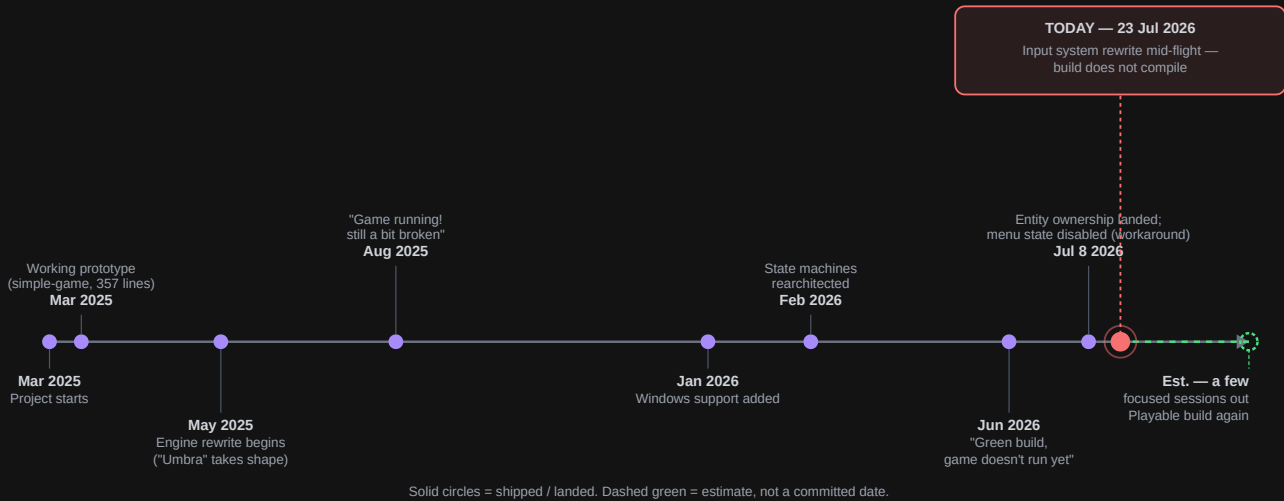
1. ~5 **concrete, mechanical compile errors** stand between here and a clean build (list below). All are small, single-file fixes.
2. **After that, the gameplay loop is still disconnected end-to-end.** Movement, collision, scoring, apple respawn, and game-over/restart all exist as *logic* somewhere in the codebase, but the object graph that would actually run them every frame was torn apart during the engine migration and never fully reassembled. The state machine that was supposed to drive menu → gameplay → game-over transitions is currently disabled outright (a deliberate workaround from the 8 July session, still in place).

Rough shape: the **engine scaffolding** (DI container, entity/component ownership, render pipeline, event bus, scene manager, systems extension point) is largely sound and well past prototype quality. The **game-specific wiring on top of it** — the part that actually makes a snake move around and eat apples — is the part that's furthest from done, because the input rework of the last two sessions deleted the old wiring faster than the new wiring landed.

Where we are in the project lifecycle

The timeline below is plotted **proportionally by real date**, not evenly spaced — that's deliberate. It makes the actual shape of this project visible: a two-week prototype in March 2025, followed by a 16-month architecture rebuild that is still mid-flight. Milestones pulled from `git log` (see each commit's date/message) and the three dev-logs.

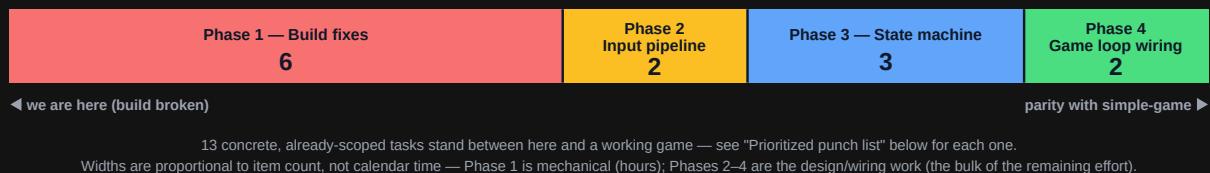
March 2025 → July 2026 (proportional by date)



Reading this left to right: the working prototype (`simple-game`) took **two weeks**. Everything since — **the other ~99% of this timeline** — has been the engine rebuild underneath it, and by its own commit messages the game has flipped between "running" and "broken" repeatedly along the way (Aug 2025, Jun 2026, and today are three separate points where a build either didn't run or didn't compile). That's not a criticism of the pace — rebuilding the architecture out from under a working prototype is genuinely slower than writing the prototype was — but it's the honest answer to "where are we in the lifecycle": **past the halfway point of the rebuild, not the project**. The scaffolding (DI, entity/component ownership, event bus, scene manager) is close to settled; the game-specific wiring on top of it is the part still actively in motion.

Remaining work, by phase

Mapping the punch list further down this report onto effort, by item count per phase (13 items total):



What "done" looks like: `origin/simple-game`

`origin/simple-game`'s `src/main.cpp` (357 lines, no engine — raylib called directly in one `main()`) is the working reference this rebuild is targeting. Its feature set, all in a single frame loop:

- Three states (menu / gameplay / game-over) switched on a plain `enum`, `Enter` to transition each way.
- WASD + arrow movement, turn-locked so you can't reverse into yourself in one frame (`directionChanged` gate).
- Grid-snapped movement paced by an accumulator (`accumulatedDistance >= snake.size`), body segments following the segment ahead of them.
- Screen-edge wraparound.

- Self-collision against every body segment → game over.
- Apple/head collision (circle check) → respawn apple at a random grid cell, `score += 10`, flag snake to grow next step.
- Score drawn every frame; game-over screen shows final score; Enter restarts.

Every one of these is a useful checkpoint for "is the engine version at parity yet" — see the table below.

Current build status

Ran a clean build (`cmake --build build`) on this branch. It fails at the **first** file in the new input pipeline, before reaching any of the game-logic files:

```
src/engine/input/input-dispatcher.cpp:42:85: error: no matching function for call to
  'InputActionPressedEvent::InputActionPressedEvent(<brace-enclosed initializer list>)'
src/engine/input/input-dispatcher.cpp:45:86: error: no matching function for call to
  'InputActionReleasedEvent::InputActionReleasedEvent(<brace-enclosed initializer list>)'
```

This is a new finding, not previously logged. The 23 July dev-log states this exact chain "compiles and is consistent end to end" — that was true of the pieces it directly traced, but not of the actual build. Root cause: `InputActionPressedEvent` / `InputActionReleasedEvent` (`engine/events/input/input-action-*.event.hpp`) publicly derive from `Core::Events::IEvent`. Under C++20 aggregate-initialization rules, a derived type with a base class needs the base listed first in the braces (`{Base{}, action}`), not just `{action}` — so `InputActionPressedEvent{action}` at `input-dispatcher.cpp:42,45` doesn't aggregate-initialize the way it looks like it should, and falls through to constructor overload resolution, which has no single- `Action` constructor. **This blocks the entire build**; nothing past this file was even reached.

Working past this file (by inspection, not by editing) surfaces further compile errors already known and logged in the 23 July log, plus one more not yet logged:

#	File:line	Problem	Logged?
1	<code>input-dispatcher.cpp:42,45</code>	Aggregate-init vs. base-class ambiguity on event construction	New — not in dev-log
2	<code>CMakeLists.txt</code>	<code>action-binding-builder.cpp</code> not in the source list; <code>action-set.cpp</code> depends on it → link failure	UE-0040
3	<code>core/input/action-value.hpp</code>	<code>ValueTypeOf</code> checks <code>is_same_v<T, bool></code> twice instead of <code>bool</code> then <code>float</code> → breaks <code>BuildKeyMap</code> 's validation assert for every axis action	UE-0039
4	<code>snake.cpp</code> (<code>Snake::Move</code>)	<code>this->inputSystem->GetActionValue(...)</code> — <code>inputSystem</code> is a reference (<code>const IInputSystem&</code>), needs <code>.</code> not <code>-></code> ; and <code>IInputSystem::GetActionValue</code> doesn't exist at all (confirmed — <code>i-input-system.hpp</code> declares nothing but a virtual destructor)	UE-0042
5	<code>snake.cpp</code> (<code>Snake::Move</code>)	Looks up <code>"MoveVertical" / "MoveHorizontal"</code> ; <code>SnakeApplication::Configure()</code> only registers <code>"MoveForward" / "MoveRight"</code> → <code>ActionSet::Get</code> asserts	UE-0043
6	<code>gameplay-scene.cpp:27-33</code> (<code>SnakeParams</code> <code>snakeParams = {...}</code>)	Designated-initializer omits <code>.inputSystem</code> and <code>.inputActions</code> — both required reference members on <code>SnakeParams</code> (<code>snake.hpp</code>) with no default. A reference member left uninitialized in aggregate-init is a hard compile error, not a runtime null	New — not in dev-log

None of these are individually hard. Together they're a half-day of mechanical fixes, in the order above (each one unblocks the compiler far enough to see the next).

After it compiles: is the gameplay loop actually wired?

This is the part worth being direct about, because it changes the honest answer to "how close are we." **No** — even with every compile error above fixed, none of the reference game's core loop would run, for a structural reason: **there are two separate `snake` / `apple` pairs, and the one that's actually driven every frame is not the one that contains the game logic.**

- `GameplayScene` (`snake-game/game-scenes/gameplay-scene.{hpp,cpp}`) owns the real, entity-managed `snake` / `apple`. `GameplayScene::Update` is what the scene manager actually calls every frame. It does exactly two things: `stateMachine.Update(deltaTime)` and `entityManager->OnUpdate(deltaTime)`. **It never calls `Snake::Update`, never checks collision, never touches score, never respawns the apple.** (UE-0034, confirmed again by direct read.)
- `GameplayState` (`snake-game/game-state/gameplay-state.cpp`) is where the reference game's actual logic lives in this codebase — apple/head collision → `score += 10` → `SetGrow(true)`, and a scaffold

for self-collision → next-state transition (currently commented out). But `GameState::snake / apple` (`gameplay-state.hpp:34-35`) are a **separate, distinct pair of members from `GameplayScene`'s**, and are never assigned by anyone — `GameState::Enter()` would hit its own `LOG_FATAL + assert` the instant it ran (UE-0014).

- It doesn't run anyway: `GameStateMachine` 's constructor has the line that would set the initial state to `MainMenuState` commented out (`gameplay-state-machine.cpp:16`), a deliberate workaround from the 8 July session so the game wouldn't segfault on a null `GameContext::input` .
`StateMachine::Update` no-ops on a null current state. So today, **no state (`MainMenuState` / `GameState` / `GameOverState`) ever runs at all** — the window opens, the entity/render pipeline ticks (you'd see the initial snake+apple drawn, static), and that's it. No menu, no game-over, no restart.
- `Confirm` / `Decline` (Enter-to-start, Enter-to-restart) are commented out in both `MainMenuState::Update` and `GameOverState::Update` — not migrated, just deleted from the build to unblock compilation (UE-0038). There is currently no code path that would read Enter as anything, even if the states were reachable.
- Apple respawn-on-pickup: `AppleSpawner::RespawnApple` (`game-systems/apple-spawner.cpp`) is actually **fully implemented** — worth correcting the record here, since the 22 July dev-log says it's "not implemented"; it compiles and has the same grid-snapped random-position logic as `simple-game` . It's just never called from anywhere (`grep` confirms zero call sites) — the apple's position is set once from an empty transform and never touched again (`gameplay-scene.cpp` 's `// MAIN QUEST: Swap this out after the apple spawner exists` comment is stale; the spawner already exists).
- Even granting all of the above got wired: `Snake::Move()` 's new input-reading code computes `vertical / horizontal` from the action system but never assigns them to `this->direction` or calls `SetDirection` (UE-0044) — so once movement input *can* be read, it still isn't applied.

Feature-by-feature vs. `simple-game`

Feature	<code>simple-game</code>	This branch, today
Build	✓ compiles, runs	✗ does not compile (5–6 fixes needed)
Menu → gameplay → game-over transitions	✓	✗ state machine disabled by standing workaround
WASD/arrow movement	✓	✗ input pipeline exists but doesn't reach <code>Snake</code> ; even once it does, direction is never applied
Turn-lock (no 180° reversal)	✓	Present in <code>SetDirection</code> 's opposite-vector check — logic survives, just unreachable
Grid-paced movement & body-follow	✓	Logic present in <code>Snake::Move / Update</code> , but <code>Snake::Update</code> has zero callers (UE-0034)
Screen wraparound	✓	Logic present in <code>Snake::Teleport</code> , same reachability problem
Self-collision → game over	✓	Scaffolded, commented out in <code>GameState::Update</code> ; also blocked on <code>GameState</code> being unreachable and its <code>snake / apple</code> being unassigned
Apple pickup → respawn + score	✓	Score/grow logic present in <code>GameState</code> ; apple respawn call exists (<code>AppleSpawner</code>) but is never invoked; both blocked on the same reachability problem
Game-over → restart	✓	<code>Confirm</code> handling commented out everywhere (UE-0038); nothing to migrate onto yet

Net: **every piece of gameplay logic from `simple-game` still exists somewhere in this codebase** — it was ported, not lost. What's missing is the wiring that makes any of it run each frame. That's arguably the more encouraging framing: this isn't a rewrite-from-scratch, it's a reassembly job.

Prioritized punch list

Ordered so each step unblocks visibility into the next; ticket IDs are the dev-log's own numbering where one exists.

Phase 1 — get a clean build

1. Fix `InputActionPressedEvent{action} / InputActionReleasedEvent{action}` aggregate-init (new finding above; `input-dispatcher.cpp:42,45`).
2. Add `action-binding-builder.cpp` to `CMakeLists.txt` (UE-0040).
3. Fix `ValueTypeOf`'s duplicated `bool` branch → should test `float` for `Axis1D` (UE-0039).

4. Fix `Snake::Move()` 's `inputSystem->` `→ .`, and resolve that `IInputSystem::GetActionValue` doesn't exist yet (UE-0042) — note the 23 July log's own recommendation to route this through `ActionBindings::BindAxis` (UE-0041) instead of patching this call site directly, once that exists.
5. Reconcile `"MoveVertical"` / `"MoveHorizontal"` vs. registered `"MoveForward"` / `"MoveRight"` (UE-0043).
6. Fill in `SnakeParams` 's missing `.inputSystem` / `.inputActions` in `gameplay-scene.cpp` (new finding above).

Phase 2 — finish the input pipeline (UE-0041) 7. Implement `ActionBindings` (currently header-only, no `.cpp`) with the `Handle`-returning RAI shape the design doc settled on, not the current `void`-returning stub — needed to avoid the two-listeners-clobber and dangling-callback-after-scene-teardown risks already identified. 8. Wire `Snake` (and `MainMenuState` / `GameOverState` for `Confirm`) through `ActionBindings` rather than touching `IInputSystem` / `ActionSet` directly (UE-0041/0038), and actually apply the read axis values to `Snake::direction` (UE-0044).

Phase 3 — reconnect the state machine (UE-0035, UE-0028, UE-0014) 9. Re-enable `MainMenuState` as the starting state in `GameStateMachine` 's constructor — this requires UE-0038 (event-driven `Confirm`) and UE-0014 (`GameState::snake` / `apple` wiring) landing together, per the existing note in the 8 July log; landing either alone just trades one crash/no-op for another. 10. Fix the use-after-move bug in `GameStateMachine` 's constructor (`StateMachine(std::move(currentState))`) then `if (!currentState)` — always true) while touching this constructor for #9 anyway (UE-0028 part 1). 11. Decide whether `GameState` (with its collision/score logic) or `GameplayScene` (what's actually ticked every frame) should own driving `Snake::Update` /collision/apple-respawn per frame — right now two parallel objects exist and only one runs. Given `GameplayScene` is what the scene manager actually calls, the simplest path may be folding `GameState` 's logic into `GameplayScene::Update` rather than trying to make `GameState` reachable at all; worth a short design decision before writing code, since it affects several tickets at once (UE-0014, UE-0034).

Phase 4 — wire the remaining game loop 12. Call `AppleSpawner::RespawnApple` on pickup (it already exists and works — just needs a call site once apple/snake collision detection is live in the frame that actually runs). 13. Wire self-collision `→ GetNextState()` /game-over transition (currently commented out in `GameState::Update`).

Lower priority / doesn't block a playable loop

- UE-0019 (`BoundaryWrapSystem` extraction — works fine inline in `Snake::Teleport` today).
 - UE-0004 (window-resize event bus stub).
 - UE-0015/0016/0020/0021/0023/0024/0026/0029/0030/0031/0032/0033 — assorted cleanups, dead members, and one still-open leak/double-free risk in `RenderComponentUIManager` (UE-0029), none of which block getting the snake moving and eating apples again.
-

Bottom line

Phase 1 is a few hours of mechanical fixes. Phase 2–4 is the real remaining work — not because any single piece is hard (the reference implementation is 357 lines and every algorithm in it already has a home in this codebase), but because the last two sessions' input-system rebuild cut the wires faster than they were reconnected, and the state machine has been sitting disabled behind a workaround since 8 July. Realistically this reads as **a few focused sessions**, not a redesign — the architecture underneath (DI, entity/component ownership, event bus, scene manager) is sound and doesn't need to change further for the game to work again.